Sibyl: Handover

Sibyl

14 October 2026

CONTENTS

Sibyl handover 3

1. What Sibyl is 3
2. Links and accounts 3
3. Run it 4
4. Repository map 5
5. How the work is recorded 5
6. Rules the code depends on 6
7. Test and release 6
8. Open work, in priority order 6
9. Known behaviours that look like bugs but are not 7

Sibyl handover

Everything needed to pick Sibyl up from release 1.0.0 (2026-10-14): what it is, where everything lives, how to run, test and release it, how the work is recorded, and what is still open.

1. What Sibyl is

Sibyl is a self-hosted control plane for an organisation's AI agents: infrastructure that lets a company safely give its agents more autonomy (D48). It keeps a registry of every agent, model and provider; watches each agent's traffic, health and drift; decides every proposed tool call before it runs (the **Action Gate**: allow, block, or escalate to a person); moves traffic to an approved fallback or a human queue when a model or provider fails; and records every decision in an audit log an auditor can verify offline. Advisory monitors (the built-in **Sentinel**, and **external monitor agents** such as a customer's own Claude, OpenAI or Gemini agent) add findings but can never enforce.

Status: **release 1.0.0 is built, tested and published.** It has not had an external security assessment, and no customer runs it yet.

2. Links and accounts

What	Where	Notes
Source (public)	https://github.com/MK-JKC/sibyl	One clean `v1.0.0` commit.
Full history (private)	https://github.com/MK-JKC/sibyl-archive	Every commit, branch and the internal working notes (discovery interviews, session handoffs). Ask the owner for access.
Live demo	https://sibyl-demo.onrender.com	Render free web service `sibyl-demo` from `render.yaml`; redeploys on every push to `main` that touches `console/`. Owned by the founder's Render account.
Demo in Codespaces	https://codespaces.new/MK-JKC/sibyl?devcontainer_path=.devcontainer/demo/devcontainer.json	Each visitor gets a private copy.
Real stack in Codespaces	https://codespaces.new/MK-JKC/sibyl?devcontainer_path=.devcontainer/real/devcontainer.json	Full stack; plug real agents in.
Release	https://github.com/MK-JKC/sibyl/releases/tag/v1.0.0	Notes come from `CHANGELOG.md`.
Images	`ghcr.io/mk-jkc/sibyl-{control,gateway,console,sim}:1.0.0`	Built by `.github/workflows/release.yml`.

There are no cloud accounts, databases or API keys behind any of this. Secrets for a real deployment are generated on the machine that runs `make up` (`deploy/compose/env-real/`, gitignored, never printed). Transfer means: GitHub repository and package ownership (or admin access), and the Render service (or redeploy it from your own Render account: **New > Blueprint > this repository**).

3. Run it

Goal	Command	Needs
Click through the product	`make demo` (http://localhost:3000)	Node.js 20.18+
Full stack with simulated agents	`make demo-stack` / `make demo-stack-down`	Docker + Compose 2.24.4+
Real deployment, your agents	`make up`, then `make status`, `logs`, `key NAME=…`, `password`, `ca`, `down [PURGE=1]`	Docker + Compose
Development checks	`make sync`, `make db-up` (PostgreSQL 17 on :5433), `make ci`	uv, Docker or a local PG17

The complete guide, with prerequisites for macOS, Linux and Windows (WSL2), ports, corporate TLS-inspection proxies, connecting agents three ways, the first Action Gate policy, external monitors, Sentinel, production hardening and troubleshooting, is `docs/setup/SETUP.md` (also `pdf/Sibyl-Setup.pdf`).

Behind a TLS-inspecting proxy: `UV_SYSTEM_CERTS=1` for uv, `NODE_EXTRA_CA_CERTS=<bundle>` for npm, and `SIBYL_EXTRA_CA_FILE` / `SIBYL_UPSTREAM_CA_HOST_FILE` for images and outbound calls. Never turn TLS verification off.

4. Repository map

Path	What it is
`services/control/`	The control plane (Python, FastAPI): `api/` (REST, 188 operations), `worker/` (scoring, alerts, failover saga, drift, Sentinel, held actions, webhooks and more; one `LoopSpec` per loop in `worker/loops.py`), `core/` (auth, crypto, settings), `migrations/` (27 plain-SQL migrations, `0000`–`0026`), `ingest/`.
`services/gateway/`	The drop-in LLM gateway: OpenAI-compatible, native Anthropic Messages and Gemini; signed policy snapshots, Action Gate on tool calls, kill switch, inline fallback, telemetry.
`packages/sibyl-sdk/`	Python SDK (Apache-2.0): `sibyl.instrument()`, `sibyl-run`, `@sibyl.tool`, framework integrations, `sibyl.monitor` client.
`packages/sibyl-schemas/`	Wire contracts (Apache-2.0): event, snapshot and policy models, canonical JSON, the secrets scrubber, golden fixtures. The fixture "secrets" are fake.
`packages/sibyl-mcp/`	The Sibyl MCP server for external monitors (13 tools).
`console/`	Next.js console with a BFF; runs `demo` (dummy data) or `live` (against the API).
`sibyl-sim/`	Test harness: provider doubles, chaos knobs, 33 scenarios, oracle and gates. Development only.
`openapi/`	The API contract, generated from `openapi/sibyl_openapi/` (`python openapi/build.py`).
`deploy/`	Docker images, Compose (`docker-compose.yml`, `.real.yml`, `.images.yml`), `real.sh`, Caddy, dex.
`ops/`	Demo launchers, backup, runbooks.
`examples/monitors/`	Ready-to-run external monitors: Claude (MCP and Agent SDK), OpenAI Agents SDK, Gemini, webhook receiver, rules script.
`ci/`	`run.sh` (the CI driver) and repository-level tests.
`.github/workflows/`	`ci.yml` (every push) and `release.yml` (on a `v*` tag).
`.devcontainer/`	The two Codespaces setups (demo and real).
`docs/`	Product, technical, sales and thesis sources (`product/`, `pdf/`, `strategy/`), setup guides (`setup/`), user guides (`guide/`), and the design record (`council/`).

5. How the work is recorded

Read these before changing behaviour; code comments cite them by number.

- **Decisions D1–D55**, the founder's authoritative answers: `docs/council/DECISIONS.md`.

- **Build rulings** (B1–B38, B-03/B-04/B-07/B-10) and lane ownership: `docs/council/build/LANES.md`. Wave specs: `build/WAVE1.md` – `WAVE4.md`, `ACTION-GATE.md`, `DRIFT.md`, `HARNESS-TIMING.md`, `SENTINEL-EVAL.md`.
- **Specifications:** architecture (`architect/ARCHITECTURE.md`), API (`api/API.md`), data model (`datamodel/DATAMODEL.md`), scoring and ML (`ml/ML.md`), roadmap with rulings R52–R71 (`roadmap/ROADMAP.md`, `roadmap/SYNTHESIS.md`).
- The original product brief: `docs/BRIEF.md`.

6. Rules the code depends on

- **No LLM or external monitor in any enforcement or detection path.** Sentinel and monitors are advisory; enforcement fails closed.
- **Row-level security on every tenant table**, and every new tenant table goes into `sibyl.erasure_cascade()`. Each migration that redefines it must carry every table (B27).
- **Migrations are append-only and contiguous** (the runner refuses gaps). The next is `0027`.
- **The API is contract-first.** Change `openapi/sibyl_openapi/`, regenerate the YAML (CI fails on drift), then update the pinned counts in `services/control/tests/contract/test_openapi_document.py` and `test_authz_wall.py`. In the console run `npm run gen:api`.
- **Strict contract scenarios** (authz matrix plus audit) for every new route.
- **Outbox readers see rows only below the cluster-wide commit horizon** (R60). Tests that read the outbox must wait for it (see `tests/worker/webhooks/conftest.py`, `tests/api/sse/test_sse.py`).
- **Never print secrets; never disable TLS.**

7. Test and release

- `make ci` runs sync, ruff, format, import contracts, mypy, pytest (parallel main suite, then serial timing tests, then the 6 harness smoke scenarios) and the licence gate. Last run (2026-10-14): **9,845 tests, 0 failures, 94 % coverage, 466 s.** It needs PostgreSQL 17 (`make db-up`). Do not run two heavy suites on one database cluster at once; open transactions hold the commit horizon and timing tests fail.
- Console: `cd console && npm run gen:api && npm run typecheck && npm run lint && npm run test:unit` (150), `CI=1 npm run test:e2e` (138 end-to-end and accessibility), `CI=1 npm run test:visual` (48; refresh with `test:visual:update`). A stale `console/.next` breaks typecheck after routes change; delete it.
- Full harness catalogue: `uv run python -m sibyl_sim run --all --parallel 4` (about 75 minutes).
- **Release:** add a `## [x.y.z] – YYYY-MM-DD` section to `CHANGELOG.md`, then `git tag -a vX.Y.Z -m "Sibyl X.Y.Z" && git push origin vX.Y.Z`. The workflow builds multi-arch images to GHCR with SBOM and provenance, and creates the GitHub Release from that section.

8. Open work, in priority order

1. **External security assessment** before any paid tenant (D18). Treat deployments as pre-production until then.

2. **Gateway budgets never refuse:** `api/snapshot/builder.py` writes `spent_usd_month: "0"` into every snapshot, so the gateway's monthly budget check never trips. Wire it to the real month-to-date spend.
3. **Design rulings for 11 harness scenarios** that fail because the specification contradicts itself or the product:
   - S-02, S-03: one leading family cannot reach the 0.60 band (B16).
   - S-04, S-08: `degraded` is unreachable under the rules/v1 tier-2 caps.
   - S-05, S-06: automated quarantine by policy is not built.
   - S-10, S-11: there is no handoff path below band `failed`.
   - S-27: the CRITICAL tier gate conflicts with D13.

   Also confirm the hardening lane's provisional rulings: R4 "contradicted status page" semantics, same-provider failover on a provider outage, B-03, R49 sample cadence, deny-holds-for-the-episode, persist-before-publish.
4. **Unbuilt features behind two scenarios:** observation-gap synthesis from heartbeat absence (S-18); prompt-injection detection with a rule-requested quarantine (S-23). One known coverage gap: masked, low-traffic agents below L5's sample minimum (S-22, B-10, strict-xfail).
5. **Console screen for drift thresholds** (the API `GET` / `PUT /api/v1/drift/thresholds` exists). Follow the `/settings/sentinel` pattern.
6. **Sentinel's judgement is unmeasured.** S-33 checks its behaviour; the real-provider evaluation (`docs/council/build/SENTINEL-EVAL.md`, about 5 USD) needs a run and sign-off.
7. **Small security items:** the SDK monitor client accepts `http://` URLs; a paused monitor's webhook deliveries accumulate until resumed; the webhook test button reports reachability of any public host; finding bodies near 32 KB can return a database error instead of 422; decide whether a monitor may see other monitors' findings.
8. **The two-tenant demo stack** shows gateway telemetry for its first tenant only (it needs a second `SIBYL_INGEST_KEYS` entry from the seed). `make demo` and real deployments are unaffected.
9. **Platform:** point-in-time recovery and off-host backups, a Helm chart, PyPI publication of `sibyl-sdk` / `sibyl-mcp` (names not reserved; install from a checkout until then), GitHub Actions pinned by SHA and moved off Node 20, hosted-runner CI time.
10. **Business:** contracting entity and IP ownership before any outreach (D46); licence choice (today the SDK and schemas are Apache-2.0 and everything else is proprietary, D15, even though the repository is public).

9. Known behaviours that look like bugs but are not

- Agents first seen in telemetry appear as `unclaimed/<hash>` until an admin confirms them; they are monitored but never routed or failed over.
- Drift alerts are watch-only and never move the health score.
- The public demo is shared by every visitor and restarts after 15 idle minutes on Render's free plan.

- `make key` does not mint monitor keys; they always belong to a registered monitor (console **Monitors > Register**).